



ELI — Installation

ELI v2.4.80

September 2026

Contents

Installation — one-click setup, cross-platform (2026-09-11)	1
Runtime OS tools (installed best-effort by <code>install.sh</code> / <code>install.ps1</code>)	1
Which download should I get? (GitHub Releases)	2
Linux — Arch and other lean distros	3
Desktop / applications-menu icons	3
CUDA toolkit option (<code>--install-cuda</code> / <code>/cuda</code>)	4
What bash <code>install.sh</code> does	4
Three local asset layers (not in git)	4
Flags	4
Files	5
Launch	5
Training dependencies now ship	5

Installation — one-click setup, cross-platform (2026-09-11)

Updated for v2.4.72 (September 2026). One hardware-aware GUI wizard for every supported OS. Primary install: GitHub Releases (ELI-Setup-*.exe, Linux AppImage, macOS dmg, portable → `./ELI_Setup.sh`). Core wizard stages hard-fail until nomic + voice + chat model are present. Windows desktop = ELI only (GUI singleton). **Home** tab ≠ Home Assistant. See first time Installation/INSTALLATION_GUIDE.md.

Runtime OS tools (installed best-effort by `install.sh` / `install.ps1`)

These are **not** Python packages — ELI installs them when a package manager is available, or offers the exact command after a failed action:

Tool	Purpose	Linux	macOS	Windows
mpv + yt-dlp	YouTube visible playback	apt/dnf/pacman + pip	brew + pip	winget mpv + pip
tesseract	Screen OCR	apt/dnf/pacman	brew	winget
xdotool / ydotool	Mouse/keyboard (X11 / Wayland)	apt/dnf/pacman	—	PyAutoGUI (pip)
grim/slurp	Wayland screenshots	apt/dnf/pacman	—	native

Tool	Purpose	Linux	macOS	Windows
playerctl	Media transport (MPRIS)	apt/dnf/pacman	brew	media keys
ffmpeg	Media/Whisper	apt/dnf/pacman	brew	winget

Wayland: ydotool needs the ydotool daemon (uinput). xdotool only reaches XWayland clients.

See docs/CROSS_PLATFORM.md for the full per-OS desktop matrix.

One command per platform sets up Python deps, the **full SQLite architecture** (blank slate — schema only, no personal data), the **nomic embedder**, and the default voice.

Platform	Command	GPU
Linux	bash install.sh --yes (--cpu-only on Iris Xe / ≤ 8 GB)	CUDA / CPU
macOS	bash install.sh --yes	Metal (no CUDA)
Windows	ELI-Setup-*.exe (frozen) or install.bat (source)	CUDA pack optional
Android	bash scripts/install_android.sh	CPU only (headless)
Portable tarball	./ELI_Setup.sh then ./RUN_ELI.sh	hardware_policy (CPU on Iris Xe / ≤ 8 GB; GPU on NVIDIA/AMD/Arc)

Which download should I get? (GitHub Releases)

Every release ships the platform assets below plus SHA256SUMS.txt.

Download	Size	Platform	What's inside	First run needs internet for	Best for
ELI-Setup-<v>.exe	~1.2 GB	Windows	Frozen Inno Setup → %LOCALAPP-DATA%\Programs\ELI	chat model	Most Windows users
ELI_v2-<v>-windows-x64.zip	~1.3 GB	Windows	ELI\ELI.exe (no Setup.bat / install.ps1)	chat model	Unzip-and-run
ELI_v2-<v>-macos-arm64.dmg	~1.4 GB	macOS (Apple Silicon)	App bundle	chat model	Mac users
ELI_v2-<v>-x86_64.AppImage	~1.2 GB	Linux	Double-click app	chat model	Most Linux users
ELI_v2-<v>-linux-portable.tar.gz	~1.2 GB	Linux	Source + ELI_Setup.sh (venv)	Python deps + chat model	Editable / CPU laptops
SHA256SUMS.txt	tiny	—	Checksums	—	verifying your download

In plain terms: - Just want it to work → **ELI-Setup*.exe (**Windows**), .dmg (**macOS**), AppImage (**Linux**). - **Want the source tree** → linux-portable.tar.gz → **./ELI_Setup.sh only (never scripts/install_eli.sh)**. - **Frozen builds bundle** nomic + Piper voice weights; **Windows TTS still**

needs a Piper CLI on PATH or under tts_piper/ (weights alone are not enough). - No download bundles a chat model. **First launch / wizard fetches one.** - **Windows: desktop shortcut is ELI only**** (Server is Start Menu). If two windows open, close one — v2.4.72+ refuses a second GUI instance.

Model size vs. your GPU matters more than which download you pick. Choose a model that fits your VRAM — e.g. **Qwen3-8B / Qwen2.5-7B** on an 8 GB GPU. Very large models (30B+) run mostly on CPU and will be slow, and the local **web server** may appear to hang on the first message while such a model loads.

Linux — Arch and other lean distros

The **AppImage** is the easiest path on any distro: it bundles its own **Python 3.11** (so Arch's system Python 3.14, which has no llama-cpp-python wheel, is irrelevant) and, since **v2.1.21**, every Qt xcb library it needs — so it launches out of the box with no extra packages. Download and run it **directly**:

```
U=https://github.com/ShadowESC95/ELI_v2.0/releases/download/v2.4.72
wget "$U/ELI_v2-2.4.72-x86_64.AppImage"
chmod +x ELI_v2-2.4.72-x86_64.AppImage
./ELI_v2-2.4.72-x86_64.AppImage
```

Two fixes worth knowing, both resolved in current builds and verified on a clean Arch VM:

- **“attempt to write a readonly database” (portable build).** If ELI is extracted onto an **NTFS / exFAT / FAT or network** filesystem — e.g. a ~/Downloads on a dual-boot data drive — SQLite's WAL journal isn't supported there and database init fails (only the user.* stores; the others in the same folder succeed, which is the tell-tale). **v2.1.19+** detects a WAL-hostile filesystem and falls back to a rollback journal automatically. On older builds, extract onto an **ext4/btrfs** path instead. Check a mount with `findmnt -T . -o TARGET,FSTYPE`.
- **GUI won't open — “libxcb-cursor0 ... xcb platform plugin”.** Qt 6.5+ needs the whole **xcb-util family**, which a minimal desktop doesn't ship (Qt's error is misleading — the real missing library is often libxcb-icccm.so.4, not the cursor lib). **v2.1.21+** bundles all of them in the AppImage. On an older build or a **source** install, add them:
 - Arch: `sudo pacman -S xcb-util-cursor xcb-util-wm xcb-util-image xcb-util-keysyms xcb-util-renderutil xcb-util`
 - Debian/Ubuntu: `sudo apt install libxcb-cursor0 libxcb-icccm4 libxcb-image0 libxcb-keysyms1 libxcb-render-util0 libxcb-util1`

Run the AppImage **directly** (as above) rather than with `--appimage-extract-and-run`, which unpacks ~4 GB into /tmp and can fail with `libz.so.1: file too short` on a small tmpfs. Launching over **SSH** into a running desktop session? Prefix with `export DISPLAY=:0`. And paste long download URLs as a **single line** — a wrapped URL 404s and its tail runs as a stray command.

Desktop / applications-menu icons

On its **first launch** the Linux AppImage offers to add **ELI, ELI Server** and **Uninstall** to your applications menu (with the app icon), pointing the launchers at the .AppImage file. This works when you run the **.AppImage directly** or via `--appimage-extract-and-run` — both set the APPIMAGE path the launcher needs. Force it any time with:

```
./ELI_v2-2.4.72-x86_64.AppImage --integrate      # add/refresh menu entries
./ELI_v2-2.4.72-x86_64.AppImage --uninstall    # remove them
```

Running the **manually extracted ./squashfs-root/AppRun** does *not* create menu icons — there's no .AppImage file for the launcher to point at, so it's skipped (with a one-line note on stderr explaining why). Use the real .AppImage for menu integration. On Ubuntu 24.04 a normal double-click/FUSE run needs libfuse2 (`sudo apt install libfuse2t64`). The **portable tarball** and **source** installs create the same menu entries via `scripts/install_desktop_apps.sh`; on **Windows** the Setup.exe adds Start-menu shortcuts, and on **macOS** the app is the .app bundle you drag to Applications.

CUDA toolkit option (--install-cuda / /cuda)

For non-technical users with an NVIDIA GPU but no toolkit. Best-effort, never fatal: - **Linux:** tries no-sudo pip nvidia-cuda-nvcc-cu12 (exposes nvcc via CUDACXX), then the system package manager (apt/dnf/pacman, if sudo is available), then prints the manual step — then source-rebuilds llama-cpp with -DGGML_CUDA=on. - **Windows:** winget install Nvidia.CUDA, then rebuilds llama-cpp. - **macOS/Android:** N/A (Metal / CPU). The default install already uses prebuilt CUDA wheels (no toolkit needed); the option only matters when those don't match the user's CUDA or a source build is required.

What bash install.sh does

1. Detects Python (3.10+) and OS; creates .venv; upgrades pip/setuptools/wheel.
2. Installs **PyTorch** (CUDA 12.1 / CPU / macOS-MPS per flags/OS).
3. Installs **llama-cpp-python** with GPU acceleration (CUDA wheel index / Metal / CPU) — then **verifies llama_supports_gpu_offload()** and, if it landed CPU-only, prints the exact CUDA-rebuild command (closes the silent-CPU-wheel trap).
4. Installs the ELI package ([full]) + all remaining dependencies from the **frozen requirements.lock.txt** (exact known-good versions; reproducible).
5. Seeds config/settings.json from the template (offline-by-default, wizard on) — never overwrites an existing config.
6. Runs **python -m eli.core.init_data** — creates **every** SQLite store/table (user.sqlite3, system_index.sqlite3, coding_memory.sqlite3, agent.sqlite3) with **zero personal memories/profile/history** (blank slate). System inventory (installed apps, \$PATH binaries) is scanned once so “open Firefox” works — that is machine environment data, not user memories.
7. Verifies import eli, the GUI entry, and the eli console script.
8. Fetches **nomic-embed-text-v1.5.Q4_K_M.gguf** → models/embeddings/ (~80 MiB, required for memory/RAG) via python -m eli.core.model_download --aux.
9. Fetches **voice weights:** Piper en_US-amy-medium + faster-whisper small.en via python -m eli.runtime.voice_assets (idempotent; skipped if already present).
10. Optionally downloads a **chat GGUF** (wizard / --model= / --auto).
11. Prints how to launch + how to add more models.

Three local asset layers (not in git)

Asset	Path	Size (typical)	When fetched
Embedder (required)	models/embeddings/nomic-embed-text-v1.5.Q4_K_M.gguf	~80 MiB	install.sh / wizard / --aux
Chat model (pick one+)	models/*.gguf	0.6-8+ GiB	Wizard / model_download / asset pack
Voice (default amy)	models/tts/piper/ + tts_piper/piper/	~60 MiB Piper + ~464 MiB whisper	voice_assets / asset pack

GitHub Release tag **local-assets-v2.1** mirrors embedder + starter chat GGUFs + cleared Piper voices. Restore: ./RUN_ELI.sh --with-github-assets. Voices excluded from auto-restore: ryan, lessac, cori — see models/MODEL_LICENSES.md.

Flags

- --cpu-only — no CUDA (CPU torch + CPU llama-cpp).
- --latest — use requirements.txt version ranges instead of the frozen lock.
- --skip-torch — leave an existing torch in place.
- --no-model — skip chat-model offer; embedder still fetched unless you disable network.

Files

- `install.sh` (Linux/macOS), `install.bat` / `install.ps1` (Windows).
- `requirements.lock.txt` — frozen exact versions (excludes torch/llama-cpp, which install via their CUDA indices).
- `pyproject.toml` — package metadata + `[project.scripts]` (`eli` → `eli.gui.app:main`).

Launch

`./eli.sh` or `source .venv/bin/activate && eli`. First run shows the **setup wizard** if no chat GGUF is present. Wizard also verifies embedder + voice and can fetch them.

Chat model: `python -m eli.core.model_download --auto` (or `--list`, or a named model). ELI stays offline by default; downloads are deliberate one-time actions.

Training dependencies now ship

`peft` and `datasets` were previously only in `requirements.lock.txt`, so a plain `pip install -r requirements.txt` produced an install whose Training tab could never leave preflight — it reported the packages missing and stopped. Both are now in `requirements.txt`, alongside:

```
peft==0.19.1
datasets==4.8.5
bitsandbytes>=0.43; platform_system != "Darwin"
```

`bitsandbytes` enables 4-bit (QLoRA) training, which lets a card too small for the full-precision weights train an adapter anyway. It is **optional everywhere** and the trainer falls back to `fp16` without it: there is no macOS wheel, and AMD needs a separate ROCm build. `install.sh` is unchanged — it already installs from the frozen lock, which carried both packages.

Optional scanners for the plugin marketplace

Neither is required, and their absence is reported rather than silently ignored:

- **ClamAV** (`clamscan` / `clamdscan` on `PATH`) — full antivirus over plugin sources.
- **yara-python** plus a ruleset at `<config dir>/plugin_yara_rules.yar`.

Without them the marketplace still runs its nine built-in engines and states that coverage was partial.

Optional runtimes for MCP servers

MCP servers are separate programs. `npm` (Node.js) and `uvx` (uv) cover almost all published servers. ELI checks for the required runtime **before** writing anything to its MCP config and names the install command for your platform if it is missing.